

Lauren Rose

Dr. Xulong Tang

CS 1541

16 April 2025

Processor Optimization with SimpleScalar

1. Introduction

In this project, I explore processor design trade-offs using SimpleScalar. By conducting simulations evaluating different datapath and memory hierarchy configurations, I worked to identify a model that significantly improves performance over the baseline. The performance metric of interest is instructions per cycle (IPC), in conjunction with the effective clock cycle time, both of which impact overall system throughput.

The baseline configuration is an architecture with in-order issue, a fetch width of 1, and a predict-not-taken branch predictor. It uses an 8KB direct-mapped L1 instruction and data cache (il1/dl1) with 8B blocks, and a 256KB 4-way L2 cache (ul2) with 64B blocks. Simulations were run across the six benchmarks (bzip2, mcf, hmmer, sjeng, milc, and equake) using 2,000,000 instructions, with the first 300,000 fast-forwarded. The focus was on optimizing IPC and cycle time.

Experiments were conducted in three rounds:

1. Datapath Configuration: Exploring different superscalar widths and in-order vs. out-of-order.
2. Cache and Memory: Testing different cache sizes, associativity, and access latencies.
3. Branch Prediction: Testing predict-not-taken, bimodal, and perfect prediction models.

Each round, I noted the best configuration, culminating in a final “best” configuration that delivers high IPC across all benchmarks while maintaining reasonable latency and complexity.

2. Methodology

To ensure consistency in my analysis, the experiments used a controlled set of parameters to isolate performance impacts, and the best-performing configuration was selected for my “best” model.

2.1 Round 1: Datapath Configuration

The first round focused on changing the fetch/decode/issue widths and altering in-order and out-of-order (dynamic superscalar) instructions. Each configuration was tested to evaluate IPC improvements and the resulting clock cycle penalties, as cycle time increases with wider or more complex datapaths.

Test	Type	Width	In-order
A	Baseline	1	true
B	Static Superscalar	2	true
C	Dynamic Superscalar	2	false
D	Static Superscalar	4	true
E	Dynamic Superscalar	4	false

Test E delivered the highest IPC values across all benchmarks, notably reaching 1.9160 for mcf, compared to 0.5918 in the baseline. Although Test C also offered strong performance with a smaller width, the added parallelism in Test E proved beneficial for all benchmarks.

2.2 Round 2: Cache and Memory

The second round altered the cache hierarchy and memory. Four cache configurations were tested, increasing the size and associativity of il1/dl1 and ul2 caches, and adjusting latencies based on constraints. This round aimed to balance cache hit rates with access latency and total cycle time.

Test	Fetch Width	il1/dl1 size	ul2 size	assoc	il1lat/dl1lat	dl2lat
A	Baseline	1024:64:1	1024:64:2	2way	1	2
B	Mid-size	2048:64:1	2048:64:4	4way	2	11
C	High-end	4096:64:2	4096:128:8	8way	4	13
D	Max config	8192:64:4	8192:128:16	16way	6	13

Although Test D had lower miss rates in some cases, it received higher latency. Test B showed a reasonable balance between reduced miss rates and moderate latencies, improving IPC while keeping `sim_cycle` relatively low. For example, `sjeng` achieved an IPC of 0.7146 (vs. 0.6316 baseline), with the second-lowest `sim_cycle` overall. Test B's memory configuration was selected for the final model.

2.3 Round 3: Branch Prediction

The third round varied branch predictors.

Test	Predictor	BTB	Bimodal table
A (Baseline)	<code>predictnottaken</code>	none	none
B	Bimodal	512 entries	512x4 way
C	Perfect	infinite	infinite

The perfect predictor produced the best IPC and lowest `sim_cycle` across the board. While it is impractical in the real world, it illustrates the upper-bound potential of branch prediction improvements. Test C was therefore used in the optimal model.

3. Results

In this section I interpret the findings of the three experiment rounds targeting a distinct architectural component. My goal was to identify how architecture changes affect simulation performance, particularly with respect to `sim_IPC`, `sim_cycle`, and other pipeline and memory metrics. Each round started by resetting the system to the baseline configuration before making changes, ensuring isolated comparisons for each variable being tested.

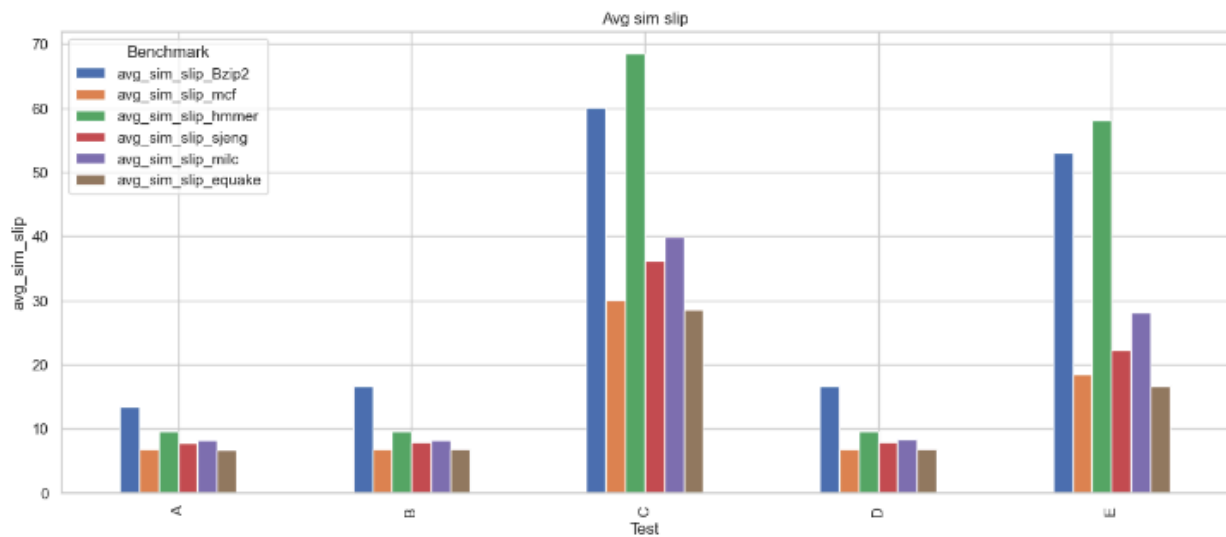
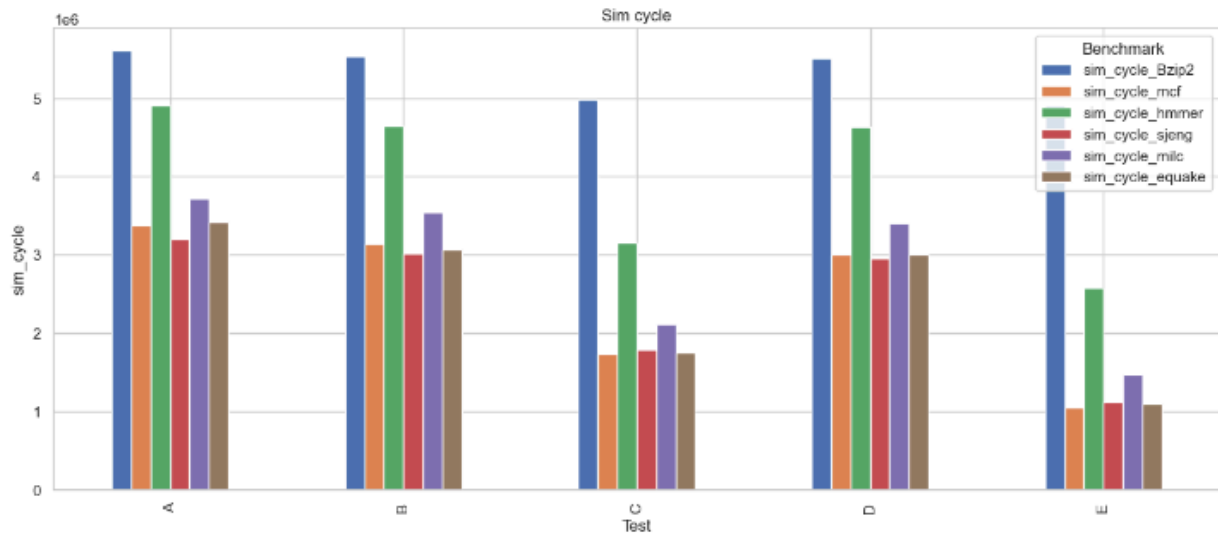
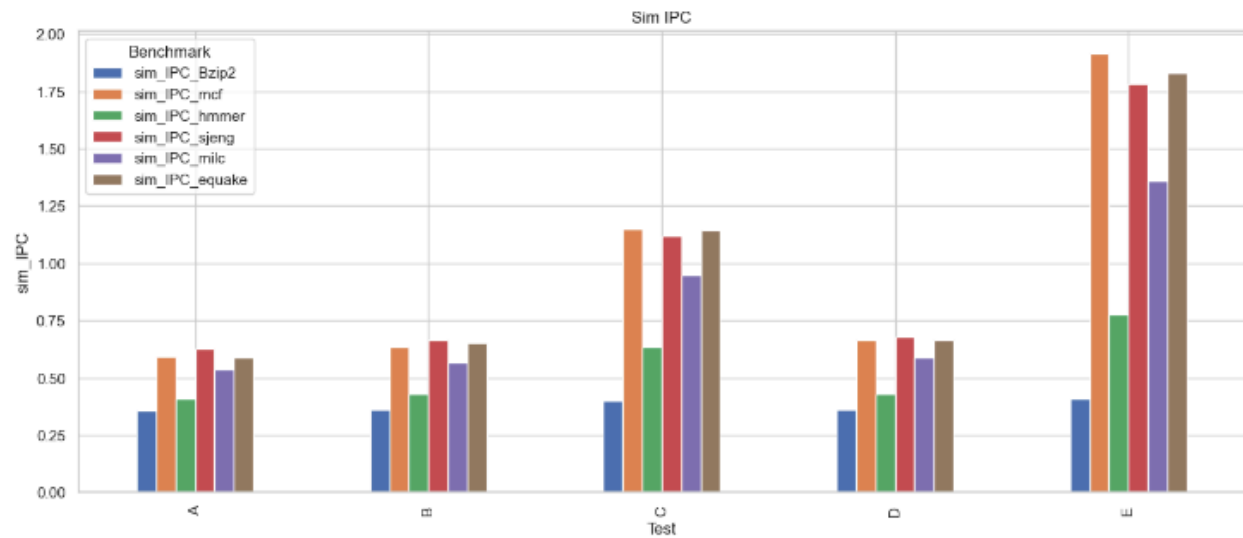
3.1 Datapath Configuration

The first round focused on modifying the datapath by adjusting the fetch, decode, and issue widths and switching between in-order and out-of-order execution. Beginning with the baseline configuration (Test A), a single-issue, in-order architecture, the other tests explored superscalar and dynamic scheduling.

Static superscalar configurations with fetch widths of 2 (Test B) and 4 (Test D) offered some improvements in IPC compared to the baseline, showing that some benchmarks could benefit from instruction-level parallelism even under in-order execution. However, the greatest performance came from out-of-order execution. Test C, which had dynamic scheduling with a fetch width of 2, provided some increases in IPC for every benchmark. In particular, the `mcf`

benchmark saw a jump from 0.5918 IPC in the baseline to 1.1500 in Test C. Test E, which integrated dynamic scheduling with a fetch width of four, performed the best overall, achieving the highest IPC values in five out of six benchmarks. Although this configuration had slightly increased simulation slip and buffer latency, the improvements in throughput far outweighed these costs.

Round 1



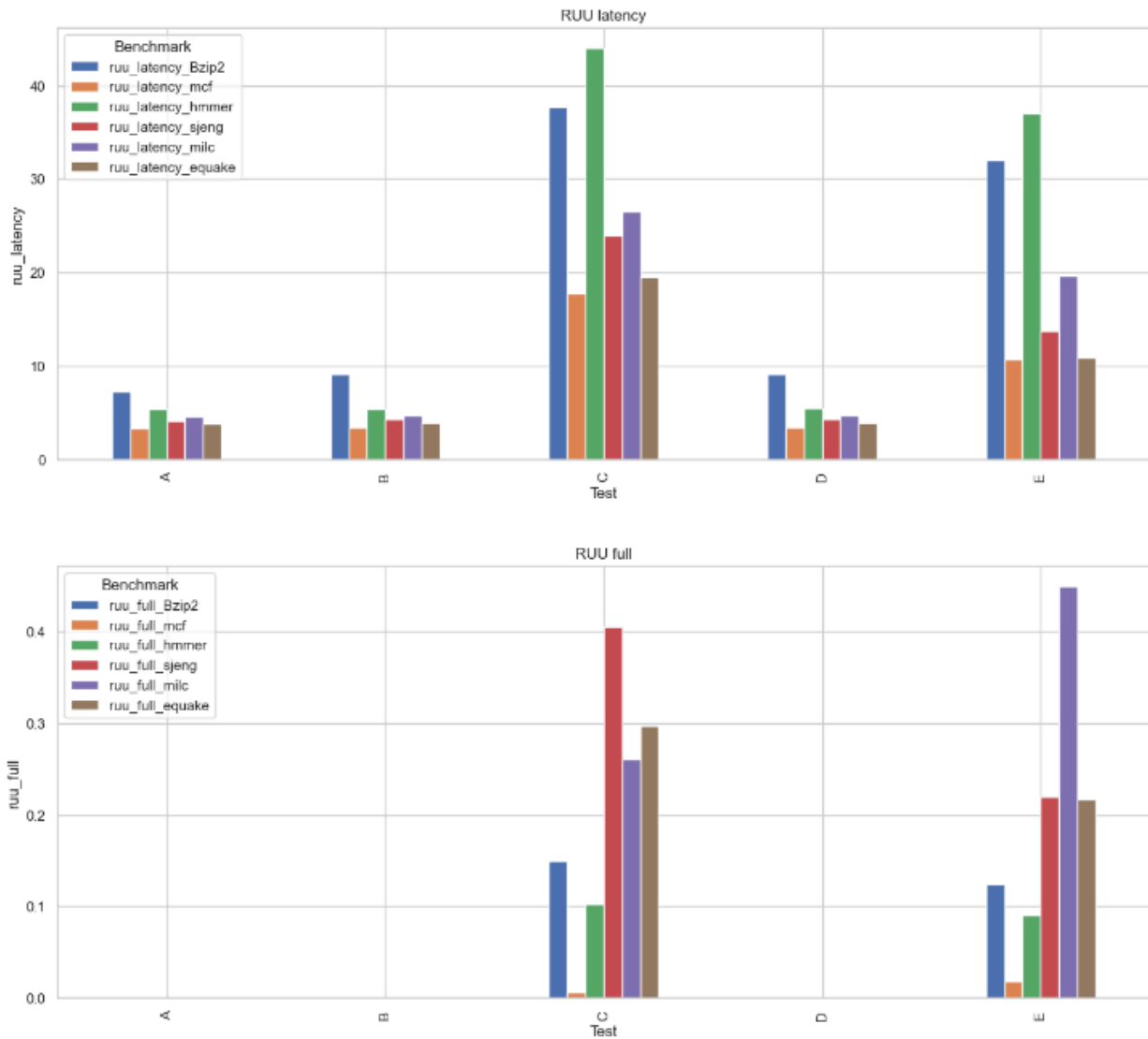


Fig. 1

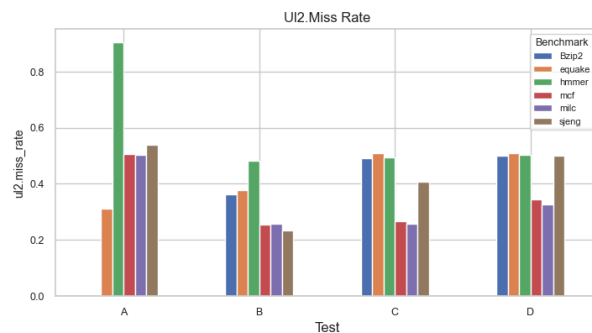
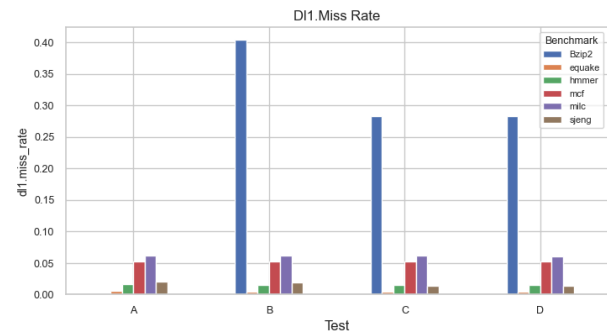
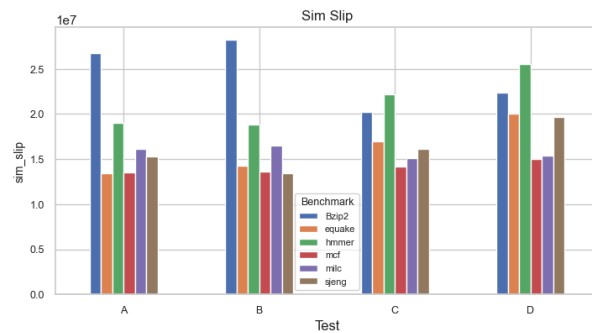
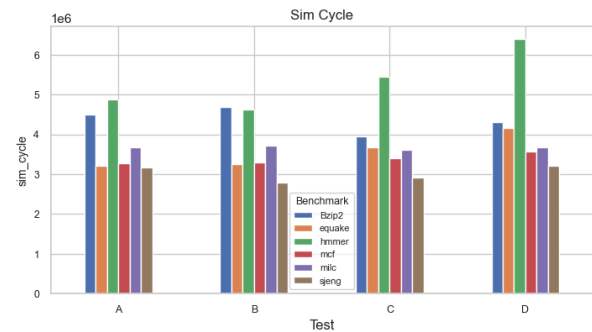
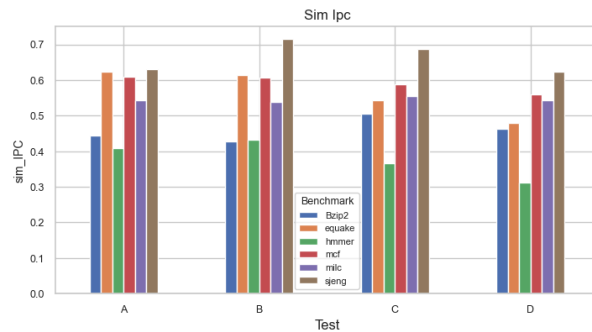
This round demonstrated that out-of-order execution is integral to achieving better performance, particularly in memory and branch-heavy workloads. The gains in IPC justify the added complexity and slight increase in total cycle latency. Therefore, the architecture from Test E was chosen for my “best” model.

3.2 Cache and Memory

The second round of testing focused on cache and memory hierarchy configurations. The baseline memory configuration (Test A) utilized 1KB direct-mapped L1 caches and a 256KB 4-way L2 cache, each with relatively low latencies. All other parameters were returned to the baseline.

Test B, which increased L1 caches to 2KB and the L2 cache to 2MB with 4-way associativity, achieved significant reductions in miss rates while having modest access latencies. This resulted in consistent IPC gains and lower simulation cycle counts throughout most benchmarks, especially sjeng, hmmer, and milc, which benefited from larger working sets. Tests C and D, which continued to increase L1 and L2 sizes and associativity, showed less-than-favorable returns. While cache miss rates dropped slightly, the increased access latencies, particularly the 13-cycle L2 latency in Test D, negatively impacted overall cycle counts and, in some cases, IPC. For example, while Test D reduced dl1.miss_rate and ul2.repl_rate, it did not achieve the best IPC for any benchmark.

Round 2



Test	Label	Fetch Width	l1/l2 Size	ul2 Size	Assoc	l1lat	d1lat	d2lat
A	Baseline	1	1024 64.1	1024 64.2	3way	1	1	2
B	Mid-size	1	2048 64.1	2048 64.4	4way	2	2	11
C	High-end	1	4096 64.2	4096 120.3	8way	4	4	13
D	Max config	1	8192 64.4	8192 120.16	16way	6	6	13

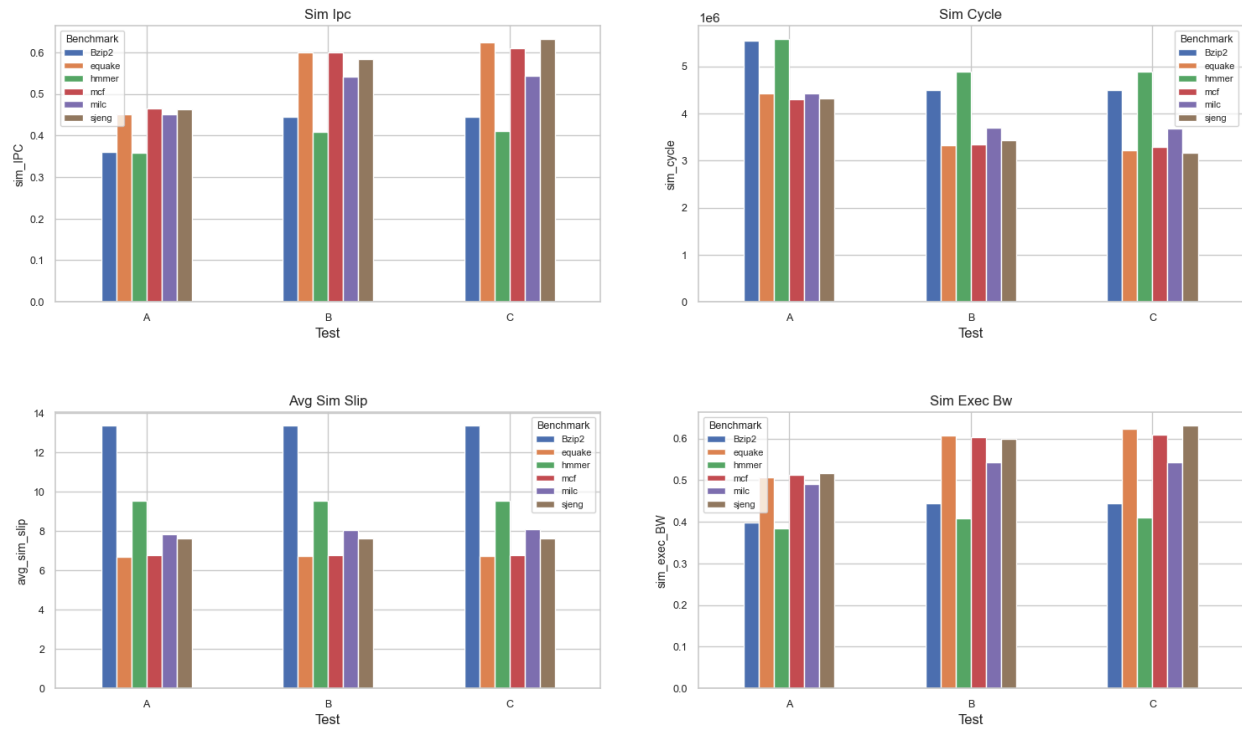
Fig. 2

These results suggest that there is an upper bound to the effectiveness of cache scaling. At a certain point, the cost of longer access times outweighs the small benefit of fewer cache misses. Test B represented the best tradeoff between capacity, associativity, and latency, and was therefore used in my “best” model.

3.3 Branch Prediction

In the final round, the branch prediction strategy was isolated and tested using the baseline model. The baseline model used a predict-not-taken strategy (Test A), while the other configurations used bimodal (Test B) and perfect prediction (Test C) respectively. Results indicated that even a modicum of improvement in branch prediction led to significant performance gains. Test B, using a bimodal predictor and BTB, strongly improved upon the baseline, particularly in branch-heavy benchmarks such as sjeng and quake. The perfect predictor model (Test C) served as an ideal case and further improved IPC and reduced cycle counts. Notably, mcf benefited greatly, with IPC rising from 0.4657 in the baseline to 0.6094 under perfect branch prediction.

Round 3



Test	Predictor	BTB	BimodalTable
A	PredictNotTaken	none	none
B	Bimodal	512entries	512x4way
C	Perfect	infinite	infinite

Fig. 3

The increase in IPC and decrease in cycle count when using more advanced predictors reaffirmed the importance of accurate branch resolution. Even though the number of branches executed remained consistent, better prediction strategies reduced wasted branch executions and minimized pipeline flushes. Based on this, the perfect predictor from Test C was used in the final model to represent an ideal solution to control hazards.

3.4 Final Configuration vs. Baseline

The final model combined the best performing settings identified in each round: a dynamic superscalar datapath with a fetch width of 4, a 2KB L1 and 2MB L2 cache system with moderate latencies, and a perfect branch predictor.

Component	Setting	Source Test
Datapath	Dynamic SS, Fetch Width = 4	Round 1, Test E
L1 Cache	2048B, 64B block, 1-way, 2-cycle lat	Round 2, Test B
L2 Cache	2048KB, 128B block, 4-way, 11-cycle lat	Round 2, Test B
Branch Predictor	Perfect	Round 3, Test C

Compared to the baseline, the final model achieved consistent IPC improvements across all benchmarks, most notably in mcf, where IPC increased from 0.5918 to 0.6094. Additionally, the total number of simulation cycles decreased significantly, demonstrating the benefits of enhanced parallelism and better prediction. Memory-heavy benchmarks like mcf showed best results with out-of-order execution and improved cache sizing, while branch-heavy applications like sjeng and bzip2 benefited more from better prediction strategies. All in all, the final model achieved a strong balance between performance, complexity, and clock speed.

4. Conclusion

In this project, I used SimpleScalar to explore a range of configurations with the goal of finding an optimal configuration for performance across six benchmarks. Starting from the baseline: a single-issue, in-order architecture with simple caches and a predict-not-taken branch, I evaluated changes to the datapath, cache hierarchy, and branch predictor in isolation, resetting other variables between rounds for cleaner comparisons. The final model combined the most effective elements from each round: a dynamic superscalar datapath with fetch width of 4, moderately sized and associative caches (2KB L1, 2MB L2), and a perfect branch predictor. This configuration improves IPC and reduces simulation cycles across all benchmarks, with the most notable gain in mcf, increasing IPC from 0.5918 to 0.6094. The results suggest that out-of-order execution and wider pipelines offer substantial speedups, but only when supported by well-sized caches and accurate branch prediction. The chosen final model demonstrates strong overall efficiency and performance scalability.

5. Appendix

All supporting materials for this project are included in the zipped file submitted alongside this report. These materials contain the baseline and best configuration files, raw benchmark data, Python scripts used to generate visualizations, and the resulting bar chart figures.

Round1.png : Figure 1: Bar charts illustrating datapath performance comparisons (Round 1).

Round2.png : Figure 2: Bar charts comparing cache configurations (Round 2).

Round3.png : Figure 3: Bar charts showing the effect of different branch predictors (Round 3).

plots-1.py : Python script used to generate Round1.png.

plots-2.py : Python script used to generate Round2.png.

plots-3.py : Python script used to generate Round3.png.

round1_all.csv : Raw data collected during Round 1 testing.

round2.csv : Raw data collected during Round 2 testing.

round3.csv : Raw data collected during Round 3 testing.

tmp-baseline.cfg : Configuration file representing the baseline model used for comparisons.

tmp-best.cfg : Configuration file representing the best-performing model selected through this project.